

Types & Hiérarchie

Hiérarchie de types Scala

- **Any** : racine de tout; **Matchable** : sous-type de Any (pattern matching); **AnyVal** : types primitifs; **AnyRef** := java.lang.Object
- **Nothing** : sous-type de tout (throw, ???), **Null** : sous-type de tout AnyRef
- **Unit** : type des effets de bord, valeur unique ()
- Types valeur : Int, Long, Double, Float, Boolean, Char, Byte, Short

Bases de Scala

- val x: Int = 5 — immuable (préféré); var x = 5 — mutable
 - Tout est une expression; le bloc retourne la dernière expression
 - Inférence de type : le compilateur déduit le type si non déclaré
- ```
val x = if condition then a else b // if = expression
val r = { val a = 1; val b = 2; a + b } // bloc
```

Interpolation de chaînes

```
s"Bonjour $name, tu as ${age + 1} ans"
f"Pi = ${math.Pi}%.4f"
raw"pas d'échappement \n ici"
```

Opérateurs numériques utiles

```
math.abs(-3) // 3
3 max 5 // 5
3 min 5 // 3
10 % 3 // 1
```

Fonctions

Définition de fonctions

```
def square(x: Int): Int = x * x
def abs(x: Int) = if x >= 0 then x else -x
```

```
// Récursion : retour obligatoire
def factorial(n: Int): Int =
 if n == 0 then 1 else n * factorial(n - 1)
```

Récursion terminale

```
import scala.annotation.tailrec
```

```
@tailrec
def factorial(n: Int, acc: Int = 1): Int =
 if n == 0 then acc else factorial(n - 1, n * acc)
```

- @tailrec : le compilateur vérifie que l'appel récursif est en position terminale → pas de stack overflow

Fonctions d'ordre supérieur

```
def apply(f: Int => Int, x: Int): Int = f(x)
```

```
// Formes équivalentes
xs.map(x => x * 2)
xs.map(_ * 2) // _ = paramètre anonyme
xs.filter(_ % 2 == 0)
xs.flatMap(x => List(x, -x))
xs.foreach(println)
```

Application partielle & Curryfication

```
def add(x: Int, y: Int) = x + y
val add5 = add(5, _) // application partielle
```

```
// Fonction curryfiée
def mul(x: Int)(y: Int) = x * y
val double = mul(2) // retourne Int => Int
```

Passage par nom (call-by-name)

```
// Le paramètre n'est évalué que si utilisé
def assert(cond: Boolean, msg: => String): Unit =
 if !cond then println(msg)
• => T : évaluation paresseuse (vs T : évaluation stricte)
```

Fonctions anonymes (lambdas)

```
val square: Int => Int = x => x * x
val add: (Int, Int) => Int = (x, y) => x + y
```

```
// Formes simplifiées
List(1,2,3).map(_ * 2)
List(1,2,3).filter(_ > 1)
List(1,2,3).reduceLeft(_ + _)
```

Collections

List

```
val xs = List(1, 2, 3)
val ys = 0 :: xs // 0 :: List(1,2,3) = List(0,1,2,3)
val empty = Nil // liste vide
val zs = xs ++ ys // concaténation
val ws = xs :+ 4 // append (moins efficace)
```

```
xs.head // 1
xs.tail // List(2,3)
```

```
xs.length // 3
xs.isEmpty // false
xs.reverse // List(3,2,1)
xs.sorted // trié
xs.distinct // sans doublons
xs.take(2) // List(1,2)
xs.drop(2) // List(3)
xs.zip(ys) // List((1,a),(2,b)...)
xs.sum // somme (si numérique)
```

Pattern matching sur List

```
xs match
case Nil => "vide"
case x :: Nil => s"un seul élément: $x"
case x :: rest => s"tête: $x, reste: $rest"
case List(a, b) => s"exactement deux: $a, $b"
// Garde
case x :: _ if x > 0 => "commence par positif"
```

Vector, ArrayBuffer, Range

```
val v = Vector(1, 2, 3) // accès O(log n), immuable
val ab = ArrayBuffer(1, 2, 3) // mutable, append O(1) amorti
ab += 4; ab -= 1 // modification en place
val r1 = 1 to 10 // Range inclusive [1..10]
val r2 = 1 until 10 // Range exclusive [1..9]
val r3 = 1 to 10 by 2 // 1, 3, 5, 7, 9
```

Tuple

```
val t = (1, "hello", true) // Tuple3[Int, String, Boolean]
t._1 // 1
t._2 // "hello"
val (a, b, c) = t // destructuration
```

Map

```
// Immuable par défaut; mutable: scala.collection.mutable.Map
val m = Map("US" -> "Washington", "CH" -> "Bern")

// Accès
m("US") // "Washington" (exception si absent)
m.get("US") // Some("Washington") – Option
m.get("XX") // None
m.getOrElse("XX", "unknown") // "unknown"

// Map totale : withDefaultValue
val m2 = m.withDefaultValue("unknown")
m2("Andorra") // "unknown"

// Modification (retourne nouvelle Map)
m + ("FR" -> "Paris") // ajout
m - "US" // suppression
m.updated("US", "NYC") // mise à jour

// Map est une fonction Key => Value
List("US","CH").map(m) // List("Washington","Bern")

// Itération
for (k, v) <- m do println(s"$k -> $v")
m.map((k, v) => (v, k)) // inverser k/v
```

Opérations avancées sur listes

map, filter, flatMap

```
List(1,2,3).map(_ * 2) // List(2,4,6)
List(1,2,3).filter(_ % 2 != 0) // List(1,3)
List(1,2,3).flatMap(x => List(x, x*10)) // List(1,10,2,20,3,30)
```

Réduction et Pliage

```
// reduceLeft/Right : pas de valeur initiale, liste non vide
List(1,2,3).reduceLeft(_ + _) // ((1+2)+3) = 6
List(1,2,3).reduceRight(_ - _) // (1-(2-3)) = 2

// foldLeft/Right : valeur initiale, liste peut être vide
List(1,2,3).foldLeft(0)(_ + _) // ((0+1)+2)+3 = 6
List(1,2,3).foldRight(0)(_ - _) // 1-(2-(3-0)) = 2
```

```
// Syntaxe alternative (Scala 2 / compatible Scala 3)
(0 /: List(1,2,3))(_ + _) // foldLeft
(List(1,2,3) :\ 0)(_ + _) // foldRight
```

Autres opérations

```
xs.exists(_ > 2) // au moins un élément > 2
xs.forall(_ > 0) // tous > 0
xs.partition(_ % 2 == 0) // (pairs, impairs)
xs.takeWhile(_ < 3) // préfixe tant que condition
xs.dropWhile(_ < 3) // suffixe après condition
xs.span(_ < 3) // (takeWhile, dropWhile)
xs.groupBy(_ % 2) // Map(0 -> pairs, 1 -> impairs)
xs.sortBy(_ .length) // tri par critère
xs.count(_ > 0) // nombre d'éléments satisfaisant
xs.find(_ > 2) // Option[A] premier élément
xs.flatten // List[List[A]] => List[A]
xs.zip(ys) // List[(A,B)]
xs.zipWithIndex // List[(A,Int)]
```

Option

Le type Option

• Représente une valeur **facultative** — remplace null de Java

• Some(x) : la valeur x existe; None : aucune valeur

```
val opt1: Option[Int] = Some(10)
val opt2: Option[Int] = None
```

```
// Tester
opt1.isDefined // true
opt1.nonEmpty // true
opt1.isEmpty // false
```

Accéder au contenu

```
// 1. get (risqué – NoSuchElementException si None)
if opt.isDefined then opt.get else default

// 2. Pattern matching (recommandé)
opt match
 case Some(x) => x
 case None => default

// 3. getOrElse (le plus concis)
opt.getOrElse(default)
```

Option + Map

```
val m = Map("US" -> "Washington")
m.get("US") // Some("Washington")
m.get("XX") // None
// Chaînage
m.get("US").getOrElse("missing") // "Washington"
// Equivalent direct
m.getOrElse("US", "missing") // "Washington"
```

Objets Fonctionnels

Classe avec constructeur primaire

```
class Rational(val x: Int, val y: Int):
 require(y != 0, "denominator must be non-zero")
 override def toString = s"$x/$y"

 def add(that: Rational): Rational =
 Rational(x * that.y + that.x * y, y * that.y)
```

• require (de Predef) lève IllegalArgumentException si faux

• val/var dans les params → accessible depuis l’extérieur

• Sans val/var → param privé au corps de la classe

Avec GCD pré-calculé

```
class Rational(x: Int, y: Int):
 private def gcd(a: Int, b: Int): Int =
 if b == 0 then a else gcd(b, a % b)
 val numer = x / gcd(x, y)
 val denom = y / gcd(x, y)
```

Hiérarchie de Classes

Trait

```
trait Animal:
 def speak(): Unit = println("...") // méthode concrète
 def move(): Unit // méthode abstraite

class Dog extends Animal:
 def move(): Unit = println("run!")
 override def speak(): Unit = println("Woof!")
```

Mixin (héritage multiple de traits)

```
// Scala 3
class Dog extends Speaker, Eater, Runner:
 def speak(): String = "Woof!"

// Scala 2
class Dog extends Speaker with Eater with Runner
```

Classe abstraite

```
abstract class Shape:
 def area: Double // abstraite
 def perimeter: Double // abstraite
 def describe = s"area=$area" // concrète

class Circle(r: Double) extends Shape:
 def area = math.Pi * r * r
 def perimeter = 2 * math.Pi * r
```

Objet singleton

```
object Logger: // créé paresseusement au premier accès
 def info(msg: String): Unit = println(s"INFO: $msg")

Logger.info("hello")
```

Objet compagnon (= static Java)

```
class Email(val address: String):
 def domain = address.split("@")(1)
 def user = address.split("@")(0)

object Email: // même nom, même fichier
 val MaxLength = 254
 def apply(user: String, domain: String): Email =
 new Email(s"$user@$domain")
 def isValid(s: String) = s.contains("@")

// Utilisation
val e = Email("alice", "example.com") // appel via apply()
Email.isValid("foo@bar.com")
```

Point d’entrée

```
// Méthode classique
object Hello:
 def main(args: Array[String]) = println("hello!")

// Scala 3 : @main
@main def birthday(name: String, age: Int) =
 println(s"Happy birthday, $name! $age years old.")
```

Packages & Imports

```
package scalaCourse.examples

import chapter9.Rational
import chapter9.{Rational, Hello} // plusieurs
import chapter9.* // tout le package

// Auto-importés : scala.*, java.lang.*, scala.Predef.*
```

Pattern Matching OO

Case class

```
trait Expr
case class Number(n: Int) extends Expr
case class Sum(e1: Expr, e2: Expr) extends Expr
case class Var(s: String) extends Expr
```

• Constructeur public automatique

• toString, equals, hashCode générés

• Immuable par défaut; copy() disponible

Pattern Matching

```
def eval(e: Expr): Int = e match
 case Number(n) => n
 case Sum(e1, e2) => eval(e1) + eval(e2)
```

Formes de motifs

```
e match
 case Number(n) => s"nombre $n" // constructeur
 case Sum(e1, _) => "une somme" // _ = wildcard
 case n: Number => "type test" // test de type
 case 0 => "zéro" // constante
 case x => s"autre: $x" // variable (minuscule)
```

• Une variable ne peut apparaître qu’une fois dans un motif

• MatchError si aucun cas ne correspond

À éviter : isInstanceOf / asInstanceOf

```
// Déconseillé – préférer pattern matching
if e.isInstanceOf[Number] then
 e.asInstanceOf[Number].n
```

OO vs Pattern Matching

- **OO decomposition** : ajouter une opération = modifier chaque classe; ajouter un type = facile
- **Pattern matching** : ajouter une opération = facile; ajouter un type = modifier tous les match

Enum & ADT

Types de données algébriques (ADT)

```
// Version case class
trait Expr
object Expr:
 case class Var(s: String) extends Expr
 case class Number(n: Int) extends Expr
 case class Sum(e1: Expr, e2: Expr) extends Expr
 case class Prod(e1: Expr, e2: Expr) extends Expr

// Version enum (équivalent, plus concis)
enum Expr:
 case Var(s: String)
 case Number(n: Int)
 case Sum(e1: Expr, e2: Expr)
 case Prod(e1: Expr, e2: Expr)
```

Enum simple (sans paramètres)

```
enum Color:
 case Red, Green, Blue
```

```
enum DayOfWeek:
 case Monday, Tuesday, Wednesday,
 Thursday, Friday, Saturday, Sunday

import DayOfWeek.*
def isWeekend(d: DayOfWeek) = d match
 case Saturday | Sunday => true
 case _ => false
```

Enum avec paramètres

```
enum Direction(val dx: Int, val dy: Int):
 case Right extends Direction(1, 0)
 case Up extends Direction(0, 1)
 case Left extends Direction(-1, 0)
 case Down extends Direction(0, -1)

 def leftTurn = Direction.values((ordinal + 1) % 4)
end Direction
```

Opérations sur enum

```
Color.Red.ordinal // 0 (index 0-based)
Direction.values // IArray de tous les cas
Direction.valueOf("Up") // Direction.Up
```

Domaine de modélisation

```
enum PaymentMethod:
 case CreditCard(kind: Card, holder: String,
 number: Long, expires: Date)
 case PayPal(email: String)
 case Cash

enum Card:
 case Visa, Mastercard, Amex
```

Expressions For

Syntaxe

```
for s yield e
// s = séquence de générateurs et filtres
// e = expression retournée à chaque itération
• Générateur : p <- e (pattern p depuis collection e)
• Filtre : if f (condition booléenne)
• Doit commencer par un générateur
```

Exemples de base

```
// Filtre + transformation
for p <- persons if p.age > 20 yield p.name

// Plusieurs générateurs (dernier varie le plus vite)
for
 x <- List(1, 2)
 y <- List("a", "b")
yield (x, y)
// => List((1,a),(1,b),(2,a),(2,b))

// Paires dont i+j est premier
for
 i <- 1 until n
 j <- 1 until i
 if isPrime(i + j)
yield (i, j)

// Produit scalaire avec zip
def scalarProduct(xs: List[Double], ys: List[Double]) =
 (for (x, y) <- xs.zip(ys) yield x * y).sum
```

Traduction en fonctions d'ordre supérieur

```
// for x <- e1 yield e2
e1.map(x => e2)

// for x <- e1 if f yield e2
e1.withFilter(x => f).map(x => e2)

// for x <- e1; y <- e2 yield e3
e1.flatMap(x => for y <- e2 yield e3)
```

Requêtes de style BD

```
case class Book(title: String, authors: List[String])

// Livres de Knuth
for b <- books; a <- b.authors
 if a.startsWith("Knuth,")
yield b.title

// Livres avec "Program" dans le titre
for b <- books if b.title contains "Program"
yield b.title

// Auteurs ayant écrit >= 2 livres
for
 b1 <- books; b2 <- books; if b1 != b2
 a1 <- b1.authors; a2 <- b2.authors
 if a1 == a2
yield a1
```

Aide-mémoire rapide

Déclarations clés

```
val x = 5 // immuable
var y = 5 // mutable
def f(x: Int): Int = x + 1 // fonction
lazy val z = compute() // évaluation paresseuse
```

Types de fonctions

```
Int => Int // fonction 1 arg
(Int, Int) => Int // fonction 2 args
=> Int // call-by-name
() => Int // thunk
```

Opérations List fréquentes

|                   |                  |               |
|-------------------|------------------|---------------|
| xs.map(f)         | xs.filter(p)     | xs.flatMap(f) |
| xs.foldLeft(z)(f) | xs.reduceLeft(f) | xs.forall(p)  |
| xs.exists(p)      | xs.partition(p)  | xs.zip(ys)    |
| xs.take(n)        | xs.drop(n)       | xs.span(p)    |
| xs.sum            | xs.min           | xs.max        |
| xs.head           | xs.tail          | xs.last       |
| xs.sorted         | xs.sortBy(f)     | xs.groupBy(f) |

Structures de contrôle

```
// if (expression)
val x = if a > b then a else b

// while (impératif)
while x > 0 do x -= 1

// for (impératif, sans yield)
for x <- xs do println(x)

// match
x match
 case 0 => "zéro"
 case n if n > 0 => "positif"
 case _ => "négatif"
```

Collections — résumé

|                  |                                |
|------------------|--------------------------------|
| List(1,2,3)      | // immuable, O(1) prepend      |
| Vector(1,2,3)    | // immuable, O(log n) accès    |
| ArrayBuffer(...) | // mutable, O(1) append        |
| Set(1,2,3)       | // immuable, sans doublons     |
| Map("k"->v)      | // immuable, key->value        |
| (1, "a", true)   | // Tuple (hétérogène)          |
| Some(x) / None   | // Option (valeur facultative) |